IMDB-RNN

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Dropout

vocab_size = 10000

(X_train, y_train), (X_test, y_test) = imdb.load_data(
    num_words=vocab_size
)
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("First review:")
print(X_train[0])

print("\nFirst review label:")
print(y_train[0])

print("\nLength of first review:")
print(len(X_train[0]))

print("Positive reviews:", np.sum(y_train == 1))
print("Negative reviews:", np.sum(y_train == 0))

max_length = 200

X_train = pad_sequences(
    X_train,
    maxlen=max_length,
    padding='post',
    truncating='post'
)

X_test = pad_sequences(
    X_test,
    maxlen=max_length,
    padding='post',
    truncating='post'
)

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)

embedding_dim = 128

model = Sequential([
    Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim,
```

```
        input_length=max_length
    ),

    SimpleRNN(64),

    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.summary()

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)

plt.figure(figsize=(8, 5))

plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)
plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')

plt.legend()
plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)

plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)

plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
plt.title('Training and Validation Loss')

plt.legend()
plt.show()

test_loss, test_accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=1
)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

predicted_labels = (predictions >= 0.5).astype(int)

print(predicted_labels[:10].flatten())
print(y_test[:10])

index = 10

review = X_test[index]
actual_label = y_test[index]

prediction = model.predict(
    np.expand_dims(review, axis=0)
)[0][0]

print("Prediction probability:", prediction)

if prediction >= 0.5:
    print("Predicted Sentiment: Positive")
else:
    print("Predicted Sentiment: Negative")

if actual_label == 1:
    print("Actual Sentiment: Positive")
else:
    print("Actual Sentiment: Negative")

word_index = imdb.get_word_index()

reverse_word_index = {
    value + 3: key
    for key, value in word_index.items()
}

reverse_word_index[0] = "<PAD>"
reverse_word_index[1] = "<START>"
reverse_word_index[2] = "<UNK>"

def decode_review(review):
    return " ".join(
        reverse_word_index.get(word, "?")
        for word in review
```

```
    )

print(decode_review(X_train[0]))

index = 0

review_text = decode_review(X_test[index])

prediction = model.predict(
    np.expand_dims(X_test[index], axis=0)
)[0][0]

print("Review:")
print(review_text)

print("\nPrediction Probability:", prediction)

if prediction >= 0.5:
    print("Predicted Sentiment: Positive")
else:
    print("Predicted Sentiment: Negative")

print(
    "Actual Sentiment:",
    "Positive" if y_test[index] == 1 else "Negative"
)
```